

Rapport projet IRGPUA

Nicolas Regnier-Vigouroux, Ernest Bardon, Grégoire Vest

October 20, 2025

Contents

1	Introduction	2
2	Version kernels from scratch	2
2.1	Reduce	2
2.2	Compact	2
2.3	Mapping	3
2.4	Histogram	3
2.5	Rapports et benchmarks	3
3	Version industrielle	4
3.1	Fix Image	4
3.1.1	Thurst	4
3.1.2	Cub	5
3.2	RMM	5
3.3	Streams	5
3.4	Rapports et benchmarks	5
4	Conclusion	6

1 Introduction

Pour ce projet, le but est d'utiliser un code implémenté sur CPU et le réimplémenter sur GPU. Deux versions de ce code optimisé doivent être faites : une version créée avec nos propres kernels en réutilisant ce que nous avons vu en TP, et une version industrielle c'est-à-dire qui pourrait être utilisée dans un cadre professionnel, en utilisant des bibliothèques existantes.

Du côté de la version kernel, on doit se concentrer sur l'optimisation du code grâce à des benchmarks/rapports et en utilisant des techniques bien spécifiques. La version industrielle se concentre plutôt sur la recherche des bibliothèques et des fonctions qui nous intéressent. En outre, les deux versions suivent la même pipeline.

Cette pipeline est décrite dans la version CPU du code et se déroule de la manière suivante : Réduction → Suppression des -27 de l'image → Correction des pixels → Égalisation d'histogramme

2 Version kernels from scratch

L'objectif de cette version est de faire un portage du programme C++ pour le CPU en Cuda pour le GPU en utilisant aucune bibliothèque, c'est-à-dire en implémentant chaque kernel à la main.

2.1 Reduce

Le reduce est basé sur l'implémentation et optimisation vu en cours. On peut voir notamment l'interleaved addressing, la correction des Warp divergences, le sequential adresssing, l'unrolling du dernier warp.

2.2 Compact

La première étape de la pipeline est d'enlever tous les -27 présent dans l'image. On va donc utiliser le pattern compact. Il est constitué de 3 parties: la création d'un masque où l'on marque les valeurs que l'on veut garder, un scan exclusif de ce masque et la répartition des bonnes valeurs aux bons index.

On a commencé par le scan car c'est le pattern le plus compliqué des 3. Nous avons décidé, car vu en cours et en TP, d'implémenter le Decoupled Look-Back Scan.

Pour effectuer qu'un seul parcours de vecteurs, plutôt que de faire un reduce du vecteur local, de regarder en arrière puis un scan de l'array local en propageant les valeurs des blocks en aval, on fait un scan du vecteur local que l'on laisse dans la shared array, on stocke la dernière valeur de notre scan pour les blocks en amont, on regarde en arrière puis on met dans le buffer final la valeur de notre scan local en ajoutant la somme local ou total des blocks en aval. Nous avons décidé de stocker et de lire nos flags de façon séquentielle malgré la perte de performance car on a eu du mal à utiliser les threadfences. On a décidé d'utiliser le scan de Kogge-Stone comme scan Block-Wise car nous avons pas eu ni le temps d'en implémenter d'autres ni de faire un benchmark complet pour trouver le plus performant pour nous.

Le Decoupled Look-Back Scan est, intrinsèquement, un scan inclusif. Pour l'utiliser en tant que scan exclusif, on a rajouté un 0 au début du masque donné en entrée du scan. Le coût de cette adaptation est juste l'allocation d'un int en plus lors de la création du masque.

Pour la création du masque, on marque avec un 1 si ce n'est pas un -27 sinon on met 0.

Pour la répartition des bonnes valeurs aux bons index, on a essayé de le faire en place mais on a eu beaucoup de mal. On s'est donc rabattue sur le faire dans un nouveau vecteur qui fait la taille de l'image final. On parcourt dans notre buffer de base dès que c'est une valeur qui n'est pas égale à -27, on met cette valeur à l'index, trouvé dans le prédicat, dans le nouveau vecteur.

2.3 Mapping

Après avoir enlevé les -27, il faut corriger chaque pixel. La correction est différente selon le modulo 4 de l'index du pixel. Il y a donc 4 corrections différentes. Sachant qu'avoir des threads qui ne font pas la même action dans un même block ralentit l'exécution du kernel, on a essayé de faire en sorte que chaque thread d'un block fasse la même action.

La première amélioration tenté fut de traiter par block les pixels qui avaient le même modulo. C'est à dire que le kernel utilise au moins 4 blocks, et les accès mémoires se font tous les 4 pixels. Mais le fait de ne plus avoir d'accès mémoires coalescents avec un stride de 4 fait perdre beaucoup plus en performance qu'avoir des opérations différentes dans des threads d'un même block donc on a abandonné cette idée.

La deuxième amélioration tenté fut d'utiliser une look-up table. Les corrections des pixels étant des additions on peut stocker le terme que l'on va ajouter au pixel à l'index correspondant. On a gardé cette technique.

2.4 Histogram

L'image est maintenant lisible mais il faut maintenant faire une égalisation d'histogramme. On alloue donc un vecteur de 256 int et on met toutes les valeurs à 0. On remplit l'histogramme en utilisant le kernel vu en cours.

Ensuite il faut faire un scan inclusif de l'histogramme, il est de taille 256, on peut donc utiliser un scan block-wise. On a utilisé un scan Kogge-Stone.

Pour trouver la première valeur non-nul de l'histogramme cumulé, on fait un atomicAdd, sur un compteur initialisé à 0, pour chaque valeur nul. La valeur finale du compteur nous donne l'index de la première valeur non nul de l'histogramme.

Il nous manque plus qu'appliquer l'égalisation sur les pixels de l'image et nous avons fini la pipeline de décodage de l'image.

2.5 Rapports et benchmarks

Nous avons pu faire un benchmark avec Nsight System grâce aux google colab. On peut voir sur la timeline ci-dessous que le travail a bien été repartie entre plusieurs streams. Le traitement de toutes les images a duré 120 ms sur les machines de google colab. Sur le rapport d'analyse, on peut voir que le kernel qui a pris le plus de temps est le Decoupled Look-Back Scan.

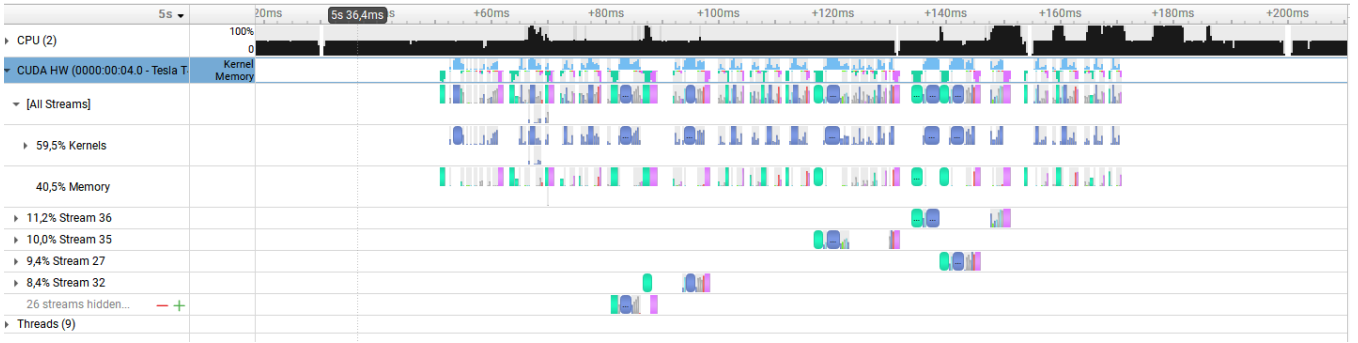


Figure 1: Timeline de la pipeline avec les kernels à la main

3 Version industrielle

L'objectif de cette version est de faire un portage du programme C++ pour le CPU en Cuda moderne en utilisant toutes les bibliothèques Cuda à disposition afin d'avoir un code à la fois simple, clair et optimisé.

3.1 Fix Image

Dans la version CPU, la fonction qui corrige les images est `fixImageCpu`. Nous avons donc utilisé les bibliothèques à disposition en suivant les étapes de cette fonction.

3.1.1 Thurst

Thurst est la bibliothèque Cuda la plus importante de cette version industrielle. C'est une puissante bibliothèque qui adapte en Cuda de nombreux concepts et algorithmes souvent utilisés en C++ afin de faciliter le travail à la fois des développeurs Cuda pour construire des algorithmes optimisés en quelques lignes et les développeurs C++ pour comprendre le code Cuda.

Tout d'abord nous pouvons utiliser `thurst::removeIf` pour enlever les valeurs -27 présents dans les images. Cette méthode à l'avantage de résumer l'étape en une ligne même si celle-ci n'est peut-être pas la plus optimisée.

Sur l'étape d'histogram equalization, les `inclusiveScan` et le `findIf` seront simplement remplacé par leur équivalent avec Thurst.

Enfin, pour l'application de l'histogram equalization nous avons utilisé `thurst::transform` avec une structure `equalize` comme transformation à appliquer sur l'image.

Après ces optimisations, il nous reste malgré tout un kernel manuel, pour la correction des pixels. Nous avons tenté d'utiliser `thurst::sequence` et `thurst::transform` afin de créer un buffer de modification à appliquer sur l'image. Cette idée n'a pas abouti à une solution viable. Nous avons donc cherché parmi les outils d'autre bibliothèques Cuda

3.1.2 Cub

Durant le projet, nous nous sommes rendus compte que Thrust n'avait pas d'outil pour faire l'histogramme de nos images. Après quelques recherches, nous avons trouvé cet outil dans la bibliothèque Cub notamment avec `cub::DeviceHistogram::HistogramEven`.

Ensuite, nous avons utilisé cette bibliothèque pour remplacer le kernel manuel pour la correction des pixels en adaptant notre idée à Cub avec `cub::DeviceFor::Bulk` et une structure `modIndexFunc`.

3.2 RMM

RMM (RAPIDS Memory Management) est une bibliothèque Cuda spécialisée dans la gestion rapide de la mémoire qui est très limitée en GPU.

Pour améliorer les performances de la version industrielle, nous avons décidé d'utiliser RMM pour tous nos buffers. Pour cela, il nous suffit de remplacer les initialisations des buffers `thrust::device` vector par `rmm::device` uvector avec `cudaMemset`. Pour le reste, l'utilisation des buffer RMM sont relativement similaires à ceux de Thrust.

3.3 Streams

Les streams sont très utiles lors de l'implémentation d'une pipeline en cuda, car ils permettent de pouvoir lancer en parallèle le traitement de plusieurs images plutôt qu'à la suite. Pour cela, nous avons utilisé le `rmm::cuda_stream_pool` qui facilite la création et la destruction des streams.

3.4 Rapports et benchmarks

Nous avons pu faire un benchmark de la version industrielle toujours avec google Colab. On y voit toujours les différents streams. On peut voir en finalité que la part de temps de calcul venant des kernels a été divisé de moitié en utilisant des kernels de bibliothèque. Le traitement de toutes les images prend 60 ms avec cette version. Elle est donc quasiment deux fois plus rapide.

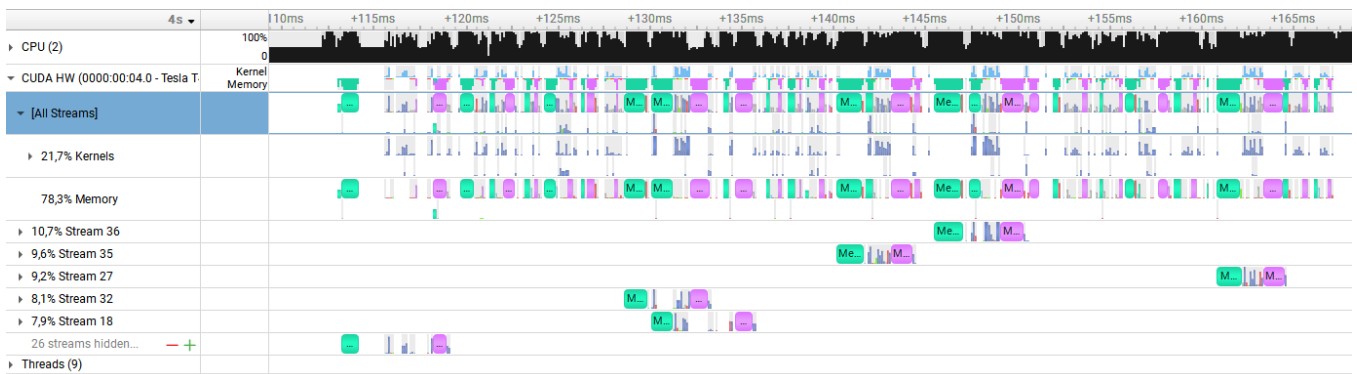


Figure 2: Timeline de la pipeline avec les kernels des bibliothèques

4 Conclusion

Pour conclure, nous avons réussi à implémenter les deux versions. Pour la première, nous avons utilisé beaucoup de techniques vues en cours et en TP, la pipeline du code donné en CPU est bien suivie et le résultat est plutôt efficace. Quant à la deuxième version, nous avons principalement utilisé Thrust et Cub car ce sont les bibliothèques qui correspondent le mieux à ce que nous cherchions. Nous avons aussi utilisé quelques outils à côté comme raft et les RMM. Le résultat de cette deuxième version est, comme nous l'avons dit précédemment, 2 fois plus efficace que la version à la main mais aussi plus simple et plus lisible ce qui correspondrait bien à une utilisation professionnelle de CUDA.

Toutefois, certaines parties de l'implémentation industrielle pourraient encore être optimisée ce qui serait une bonne piste d'amélioration à l'avenir.